StatCore Nearest Neighbours: Exact Baselines, Prototype Compression, Diagonal Metric Learning, and Data-Adaptive k

Jabulani Monchwe
Applied Business Analytics and Decision Making
North-West University
IDEA-Research Club Chairperson
Johannesburg, South Africa
monchwej.25@gmail.com

Martin Chanza
Applied Business Analytics and Decision Making
North-West University
Mafikeng, South Africa
martin.chanza@nwu.ac.za

Olorato Motitswane
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
oloratomotitswane@gmail.com

Mfundo Monchwe
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
diditmfundo@gmail.com

***Abstract*—This paper specifies the pure-NumPy StatCore k-nearest-neighbour (KNN) classifier and regressor as implemented, including their common standardisation and voting rule and the complete `base`, `prototype_compressed`, `learnable_metric`, and `stat_k_selection` branches. The method-specific names denote concrete implementation choices rather than a claim of novelty or general superiority. In particular, the metric-learning branch uses a diagonal nonnegative metric and, despite the intended neighbourhood-components interpretation, includes each sampled observation itself among its soft neighbours; the selection branch is exact leave-one-out scoring but standardises once on the complete training set. These details matter for interpreting both the gradients and the validation evidence. Reproducible experiments compare all StatCore variants with scikit-learn KNN as a standard reference on nonlinear, noisy, irrelevant-feature data and the bundled Breast Cancer Wisconsin and Diabetes datasets over three predefined splits. We report accuracy, log loss, equal-width expected calibration error (ECE), regression losses, and elapsed fit-plus-prediction time, and explicitly retain cases in which the variants do not improve on the standard baseline.**

***Index Terms*—nearest neighbours, prototype compression, diagonal Mahalanobis distance, metric learning, model selection, reliability**

I. Introduction

Nearest-neighbour prediction is a nonparametric method whose decisions depend directly on a representation, a distance, a neighbourhood size, and an aggregation rule [1], [2]. StatCore supplies a NumPy-only implementation for classification and scalar regression. Its variants alter four distinct components: the reference set, the distance geometry, or the integer neighbourhood size. This paper's contribution is an implementation-level mathematical specification and a reproducible finite benchmark, not a new consistency result or a universal performance claim.

The exact source of record is `src/ml_reliability/statcore/knn/knn.py`; shared validation, standardisation, and distance helpers are in `statcore/_common.py`. The package contract is recorded in `statcore/SPEC.md`. Scikit-learn appears only in the experiment script as a standard baseline and as a source of bundled datasets; all StatCore estimators and proposed variants are NumPy implementations.

II. Notation and Common Implementation

Let $X \in \mathbb{R}^{n \times d}$ and $y = (y_i)_{i=1}^{n}$. The classifier obtains sorted unique labels $\mathcal{C} = (c_1, \ldots, c_C)$ using `numpy.unique`, encodes each target as one-hot $t_i \in \{0,1\}^C$, and requires $C \geq 2$. The regressor requires finite numeric scalar responses and stores $t_i = y_i$ as a one-column matrix. Inputs are converted to finite floating point, must be two-dimensional with at least one feature, and training requires at least two observations. Prediction checks the fitted flag and feature count.

For each feature j, the scaler stores the training mean $\mu_j = n^{-1}\sum_i x_{ij}$ and population standard deviation $s_j = \sqrt{n^{-1}\sum_i (x_{ij} - \mu_j)^2}$. It substitutes 1 when $s_j \leq 10^{-12}$. Thus $z_{ij} = (x_{ij} - \mu_j)/\max(s_j, 1 \text{ if } s_j \leq 10^{-12})$. The pairwise squared distance is evaluated using norms and matrix

multiplication, then clipped below at zero to suppress small negative round-off:

$$d^2(z, z') = \max\{\|z\|_2^2 + \|z'\|_2^2 - 2z^\top z', 0\}. \tag{1}$$

This scaler is fitted once on all fit-time rows, before any KNN-specific selection. At inference, query vectors use the saved training scaler.

For query q, let $\mathcal{N}_k(q)$ be the first $\min(k, r)$ indices in a stable ascending sort of squared distances to r references. If the stored reference weight for reference j is $v_j > 0$, the unnormalised vote weight is

$$u_{qj} = \begin{cases} v_j, & \texttt{weights=uniform}, \\ v_j/(\sqrt{d^2(q, z_j)} + 10^{-9}), & \texttt{weights=distance}, \end{cases} \tag{2}$$

$$\alpha_{qj} = u_{qj} / \max\left(\sum_{\ell \in \mathcal{N}_k(q)} u_{q\ell}, 10^{-300} \right). \tag{3}$$

The prediction vector is $\hat{t}_q = \sum_{j \in \mathcal{N}_k(q)} \alpha_{qj} t_j$. Class probabilities are these normalized class votes; they are not passed through a softmax and are not calibrated by a separate procedure. Class labels use the first maximum in sorted `classes_`. Regression returns the sole target component. If k exceeds the reference count, it is silently capped. With zero-distance duplicates, distance weights remain finite due to 10^{-9}; a zero-feature-variance column contributes zero after scaling.

III. Reference method: standard KNN

The `base` variant stores every scaled training row and its target, sets the diagonal metric to $\mathbf{1}_d$, and uses the constructor's positive integer `n_neighbors` (default $k = 5$). It supports `weights=uniform` (default) and `weights=distance`. This is the standard local vote/mean rule in (3), not a learned bandwidth or a probabilistic generative model. Its inductive bias is local smoothness in standardized Euclidean coordinates; irrelevant or differently informative features can distort neighbourhoods despite marginal standardisation.

IV. Implemented own variants

A. Per-class prototype compression

For `variant=prototype_compressed`, let $\rho = \texttt{compression} \in (0, 1]$ (default 0.2). In classification, each class c is clustered separately in scaled feature space. For class index set I_c, the requested cluster count is $m_c = \max(1, \lceil \rho |I_c| \rceil)$. If $m_c \geq |I_c|$, the class's original observations are retained with unit reference weights. Otherwise, the implementation runs StatCore KMeans with `variant=better_init`, `n_init=2`, `max_iter=50`, and the supplied `random_state`. For each nonempty cluster G, it stores the feature centroid $|G|^{-1} \sum_{i \in G} z_i$, mean one-hot target $|G|^{-1} \sum_{i \in G} t_i$, and reference weight $v_G = |G|$. Weighting the centroid target by its cluster size preserves the cluster's total class mass under uniform votes, although it does not make the compressed geometry equivalent to the original sample geometry.

For regression, all rows form one cluster partition. KMeans is run in the augmented space $[z_i, \tilde{y}_i]$, where $\tilde{y}_i = (y_i - \bar{y})/(s_y + 10^{-12})$ and s_y is the population standard deviation. Only the z coordinates of each cluster centroid are retained as reference features; its target is the cluster's mean original-scale y, and its reference weight is cluster size. This target-informed clustering can alter both the locations and target averages, and can be optimistic if assessed on data that influenced prototype construction. Prediction then applies (3) over the reduced references, with cluster-size multiplicity represented by v_G.

B. Learnable diagonal metric

The `learnable_metric` branch retains all training rows but learns a positive diagonal scaling. To bound computation, a seeded random permutation selects $m = \min(n, 400)$ observations. Write their standardized features as z_i and squared coordinate differences as $D_{ij\ell} = (z_{i\ell} - z_{j\ell})^2$. The parameters are log-weights θ_ℓ, initialized to $\log(1/d)$, and $a_\ell = \exp(\theta_\ell)$. The soft neighbour distribution is

$$P_{ij}(a) = \frac{\exp(-\sum_{\ell=1}^d a_\ell D_{ij\ell})}{\sum_{r=1}^m \exp(-\sum_{\ell=1}^d a_\ell D_{ir\ell})}, \quad E_{i\ell} = \sum_j P_{ij} D_{ij\ell}. \tag{4}$$

There is no diagonal mask in this calculation: $D_{ii\ell} = 0$ and the query itself is one of its candidates. In classification, $S_{ij} = \mathbb{1}[y_i = y_j]$, $q_i = \sum_j P_{ij} S_{ij}$, and the minimized objective is $L_{\text{cls}} = -m^{-1} \sum_i q_i$. Differentiating the softmax gives

$$\frac{\partial P_{ij}}{\partial a_\ell} = P_{ij}(-D_{ij\ell} + E_{i\ell}), \tag{5}$$

$$\frac{\partial q_i}{\partial a_\ell} = -\sum_j P_{ij} S_{ij} D_{ij\ell} + q_i E_{i\ell}, \tag{6}$$

$$\frac{\partial L_{\text{cls}}}{\partial a_\ell} = m^{-1} \sum_i (A_{i\ell} - q_i E_{i\ell}), \quad A_{i\ell} = \sum_j P_{ij} S_{ij} D_{ij\ell}. \tag{7}$$

This is exactly the implemented gradient. Since $\partial a_\ell / \partial \theta_\ell = a_\ell$, the log-coordinate gradient is $g_{\theta,\ell} = a_\ell \partial L / \partial a_\ell$.

For regression, responses in the sampled subset are standardized as $r_i = (y_i - \bar{y}_s)/(s_s + 10^{-12})$. The soft estimate is $\hat{r}_i = \sum_j P_{ij} r_j$, and the loss is $L_{\text{reg}} = m^{-1} \sum_i (\hat{r}_i - r_i)^2$. Its derivative is

$$\frac{\partial \hat{r}_i}{\partial a_\ell} = -\sum_j P_{ij}(r_j - \hat{r}_i) D_{ij\ell}, \tag{8}$$

$$\partial_{a_\ell} L_{\text{reg}} = \frac{2}{m} \sum_i (\hat{r}_i - r_i)\, \partial_{a_\ell} \hat{r}_i. \tag{9}$$

The implementation applies Adam to θ, with $\beta_1 = 0.9$, $\beta_2 = 0.999$, bias correction, $\epsilon = 10^{-8}$, and learning rate `metric_lr` (default 0.1); after every update it clips each θ_ℓ to $[-8, 3]$. It performs `metric_iters` iterations (default 60), with no objective regularizer or validation split. Finally a is normalized by its arithmetic mean, $a \leftarrow a/\bar{a}$, and prediction distances use $\sum_\ell a_\ell (z_\ell - z'_\ell)^2$. Normalization fixes the arbitrary global distance scale for the finite-temperature softmax, though

here inference uses hard neighbours. The implementation is a diagonal, in-sample soft-neighbour heuristic, not full-matrix NCA or large-margin metric learning.

The self-candidate is consequential: it contributes a zero-distance, same-label term in classification and the exact target in regression, making the sampled training objective easier than a leave-one-out objective. The docstring's intended leave-one-out description is therefore not what this code computes. No claim of generalization follows from minimizing this self-inclusive objective.

C. Leave-one-out statistical k selection

For `variant=stat_k_selection`, the estimator constructs a full $n \times n$ squared-distance matrix after standardizing all n fit-time rows, then sets its diagonal to $+\infty$. It uses unit reference weights and evaluates candidates using exact leave-one-out neighbours, without a fold-specific scaler. The default candidate list is the odd integers $1, 3, \ldots$ not exceeding $\min(31, n-1)$. A user-supplied `k_grid` is converted elementwise to integers and filtered to $1 \le k \le n-1$; an empty valid list raises `ValueError`. For classification,

$$J_{\mathrm{cls}}(k) = 1 - \frac{1}{n}\sum_i \mathbb{1}[\arg\max_c \hat{p}_{ic}(k) = y_i] \tag{10}$$

$$+ 10^{-3}\frac{1}{n}\sum_i -\log\big(\max\{\hat{p}_{i,y_i}(k), 10^{-3}\}\big)\,. \tag{11}$$

where $\hat{p}_{ic}$ follows the selected uniform or distance weighting. For regression, $J_{\mathrm{reg}}(k) = n^{-1}\sum_i(\hat{y}_i(k) - y_i)^2$. The first candidate attaining the minimum is selected; a later score must improve by more than 10^{-12} to replace it. Scores are retained in `cv_scores_`; $k_\star$ replaces `n_neighbors` for prediction on the complete training reference set. This is a validation heuristic over the training sample, not an unbiased external estimate. In particular, the global training scaler has seen every candidate row, albeit without labels.

V. Algorithms

The following compact procedures make branch order explicit. Lines named "shared" apply to both tasks; task-specific targets are specified above.

A. Fit and prediction

1) Validate finite numeric X, aligned one-dimensional y, at least two training rows, positive integer k, $0 < \rho \le 1$, and a recognized variant/weighting.
2) For classification, set sorted `classes_` and one-hot T; reject a single class. For regression, check and cast the scalar targets.
3) Fit the population-statistic scaler on X, transform to Z, initialize $a = \mathbf{1}$ and $k_\star =$ `n_neighbors`.
4) If `learnable_metric`, run the seeded diagonal-metric procedure in Section IV-B; transform stored features by $\sqrt{a}$.
5) If `stat_k_selection`, run the diagonal-masked LOO criterion in Section IV-C and replace $k_\star$ with its minimizer.
6) If `prototype_compressed`, perform task-specific clustering and replace (Z, T) by nonempty prototype means with their cluster-size weights; otherwise retain every observation with unit weights.
7) Store references, targets, weights, a, $k_\star$, feature count, and fitted flag. For a query, validate dimensions, scale and apply $\sqrt{a}$, compute all reference distances, stably sort, and return the normalized weighted target average (3).

B. Metric optimization

1) Draw a seeded random permutation and retain its first $m = \min(n, 400)$ rows; build $D_{ij\ell}$ and initialize $\theta_\ell = \log(1/d)$, first moments $u = 0$, second moments $v = 0$.
2) At each $t = 1, \ldots,$ `metric_iters`, compute $a = \exp(\theta)$, scores $-\sum_\ell a_\ell D_{ij\ell}$, their stabilized softmax P, and E from (4); do not exclude the self entry.
3) Calculate the classifier gradient (7) or regressor gradient (9), then chain it to θ by elementwise multiplication with a.
4) Update $u_t = 0.9u_{t-1} + 0.1g_t$, $v_t = 0.999v_{t-1} + 0.001g_t^2$, bias-correct both, and set $\theta \leftarrow \mathrm{clip}(\theta - \eta\hat{u}/(\sqrt{\hat{v}} + 10^{-8}), -8, 3)$.
5) Return $a/\operatorname{mean}(a)$ and use the resulting diagonal metric for the hard-neighbour stage.

VI. Assumptions, Complexity, and Edge Cases

The constructor defaults are `n_neighbors=5`, `variant=base`, `weights=uniform`, `compression=0.2`, `k_grid=None`, `metric_iters=60`, `metric_lr=0.1`, and `random_state=0`. The default candidate grid is defined in Section IV-C. The code explicitly checks the variant, weight name, compression interval, and positive-integer neighbor count at fit; it does not range-check metric iterations or learning rate. A user-supplied grid is integer-converted and filtered only for statistical selection, and raises only when no entries remain valid.

All algorithms assume rows are exchangeable for the benchmark split and that Euclidean proximity after marginal scaling is meaningful. The diagonal metric cannot represent feature interactions or rotations. The learned-metric loss is self-inclusive and optimized on the training subset; the selected-k score is training-set LOO but does not refit preprocessing within each held-out fold. Prototype compression changes the support and can bias boundaries or smooth nonlinear regression. Distance-weighted estimates can be dominated by a near-duplicate; the 10^{-9} term prevents division by zero but is not an uncertainty model. Class ties resolve to the first sorted class. A request for k larger than the available references uses all references; selected k is always at most $n - 1$ when the default or filtered grid is used. Constant features are preserved at zero scaled value. Inputs with NaN or infinity, dimension mismatch, nonpositive or noninteger initial k, invalid compression, invalid weighting, one-class classification, or an empty valid k grid fail explicitly.

TABLE I
VARIANT COMPARISON, WITH IMPLEMENTATION-SPECIFIC OPERATIONS.

Method	Classifier	Regressor	Extra computation / principal caveat
Standard reference	`base`: scaled Euclidean neighbours; class vote	`base`: scaled Euclidean neighbours; target average	Stores n references; feature relevance is not learned
Prototype compression	Classwise KMeans; centroid targets and cluster-count weights	KMeans on scaled X plus standardized target; retain feature and target means	Reduced inference set; cluster approximation can erase local structure
Learnable metric	Maximize self-inclusive soft same-class mass under diagonal metric	Minimize self-inclusive soft prediction MSE under diagonal metric	At most 400 samples, 60 Adam steps by default; not true LOO training
Statistical k selection	Choose k by LOO error plus 10^{-3} clipped log-loss	Choose k by LOO MSE	$O(n^2)$ distance storage; preprocessing remains full-sample

For query batch size q, r stored references, and d features, distance computation is $O(qrd)$ time and $O(qr)$ distance storage; full stable sorting costs $O(qr \log r)$, plus $O(qkC)$ vote aggregation. Standard fit stores $O(nd)$ features and $O(nC)$ targets (or $O(n)$ scalar targets). Prototype clustering uses implementation-dependent KMeans iterations; storage after compression is $O(rd+rC)$ with $r \leq n$ and often substantially smaller. Metric learning explicitly materializes $D \in \mathbb{R}^{m\times m\times d}$ and repeatedly performs $O(m^2d)$ operations, thus $O(m^2d)$ memory and $O(Tm^2d)$ time for T iterations. Although $m \leq 400$, large d still incurs cubic-in-m-times-d-scale storage. LOO k selection stores $O(n^2)$ distances and invokes a stable sort for each candidate, costing approximately $O(n^2d + |\mathcal{K}|n^2 \log n)$.

VII. EMPIRICAL EVALUATION

A. Protocol

The script docs/latex/experiments/knn_experiment.py (with `_experiment_utils.py`) is the executable specification of this experiment. From the repository root run `python docs/latex/experiments/knn_experiment.py` using the project venv. It evaluates classification and regression separately on (i) a 640-row nonlinear synthetic sample with two informative coordinates, 18 independent Gaussian irrelevant features, label flips at probability 0.10 or Gaussian response noise with standard deviation 0.30, and (ii) the bundled scikit-learn Breast Cancer Wisconsin classification or Diabetes regression data. Each dataset uses 75/25 train/test splits for seeds 11, 23, and 47. The synthetic task is regenerated per seed. Splits are generated in NumPy: the test count is $\lceil 0.25n \rceil$; classification allocates proportional per-class counts by floors plus largest fractional remainders (seeded tie-breaking), permutes within each class, and shuffles the combined partitions; regression uses a seeded permutation. Every reported cell is the mean and sample standard deviation over these three runs.

Scikit-learn is used strictly for the standard comparison estimator (scaled uniform $k = 5$ KNN) and bundled dataset access. The StatCore comparisons are its `base` and all three own variants. All methods use uniform votes in this benchmark. Own-variant settings are $\rho = 0.35$, 40 metric steps, learning rate 0.1, and the default k grid; the benchmark's fixed `n_neighbors=5` is used only where applicable. Classification metrics are accuracy, natural-log cross-entropy (probabilities clipped and row-normalized for metric calculation), and 15-bin equal-width ECE, $\sum_b (n_b/N)|\operatorname{acc}(b) - \operatorname{conf}(b)|$. Regression reports MSE, MAE, and R^2. Each estimator receives one untimed warm-up fit and test prediction on that same training split; runtime is the subsequent wall-clock fit plus test prediction, excluding dataset generation and metric aggregation. No hyperparameter is chosen using the test set; results characterize these fixed settings and splits rather than an inferential population guarantee.

B. Classification results

Table II records the measured values from the script. A cell is mean $\pm$ sample standard deviation; time is seconds. Values are intentionally not generalized beyond the datasets and seeds.

C. Regression results

Observed differences should be read alongside these limitations: only one fixed split per seed is used; ECE is bin-dependent and noisy on small test sets; KNN probabilities are vote fractions rather than calibrated likelihoods; and the self-inclusive metric objective is a training objective, not held-out evidence. Prototype reduction trades reference memory and query cost against an approximation. No method is assumed to win on every task; the tables preserve negative as well as positive comparisons.

VIII. REPRODUCIBILITY AND REPORTING LIMITATIONS

The scripts print the exact mean and sample SD rather than embedding generated or hand-selected numbers in code. They use the installed NumPy and scikit-learn versions, a fixed seed list, deterministic model random states, and fixed hyperparameters. Runtime is hardware-, BLAS-, and load-dependent. The bundled datasets are small and do not constitute a broad benchmark; these experiments do not quantify uncertainty over datasets, provide nested tuning, test temporal or group shifts, or establish calibration under deployment shift. Regression

TABLE II
CLASSIFICATION: ACCURACY, LOG LOSS, 15-BIN ECE, AND FIT-PLUS-PREDICT TIME (MEAN ± SD, THREE SEEDS).

Dataset	Method	Accuracy	Log loss	ECE	Time (s)
Synthetic	sklearn standard	0.5646 ± 0.0452	1.2460 ± 0.1257	0.1254 ± 0.0423	0.0116 ± 0.0019
Synthetic	StatCore base	0.5646 ± 0.0452	1.2460 ± 0.1257	0.1254 ± 0.0423	0.0060 ± 0.0012
Synthetic	prototype compressed	0.5750 ± 0.0375	0.8103 ± 0.0511	0.1206 ± 0.0382	0.0473 ± 0.0126
Synthetic	learnable metric	0.5583 ± 0.0253	1.8623 ± 0.6171	0.1517 ± 0.0166	0.6717 ± 0.1421
Synthetic	statistical k selection	0.5792 ± 0.0940	0.6730 ± 0.0346	0.0874 ± 0.0298	0.2673 ± 0.0156
Breast cancer	sklearn standard	0.9464 ± 0.0246	0.2255 ± 0.1370	0.0219 ± 0.0127	0.0143 ± 0.0028
Breast cancer	StatCore base	0.9464 ± 0.0246	0.2255 ± 0.1370	0.0219 ± 0.0127	0.0057 ± 0.0007
Breast cancer	prototype compressed	0.9510 ± 0.0140	0.3617 ± 0.1152	0.0397 ± 0.0176	0.0754 ± 0.0146
Breast cancer	learnable metric	0.9324 ± 0.0225	0.1153 ± 0.0147	0.0289 ± 0.0082	0.8329 ± 0.0709
Breast cancer	statistical k selection	0.9464 ± 0.0246	0.3372 ± 0.1190	0.0246 ± 0.0102	0.2080 ± 0.0442

TABLE III
REGRESSION: MSE, MAE, R^2, AND FIT-PLUS-PREDICT TIME (MEAN ± SD, THREE SEEDS).

Dataset	Method	MSE	MAE	R^2	Time (s)
Synthetic	sklearn standard	0.5789 ± 0.0813	0.6128 ± 0.0563	0.2957 ± 0.0229	0.0123 ± 0.0032
Synthetic	StatCore base	0.5789 ± 0.0813	0.6128 ± 0.0563	0.2957 ± 0.0229	0.0060 ± 0.0014
Synthetic	prototype compressed	0.5650 ± 0.1034	0.6063 ± 0.0555	0.3135 ± 0.0715	0.0894 ± 0.0515
Synthetic	learnable metric	0.1383 ± 0.0218	0.2980 ± 0.0295	0.8279 ± 0.0470	0.6512 ± 0.2266
Synthetic	statistical k selection	0.5402 ± 0.0684	0.5929 ± 0.0463	0.3421 ± 0.0103	0.2211 ± 0.0449
Diabetes	sklearn standard	4028.9521 ± 366.8942	50.4348 ± 1.0521	0.3095 ± 0.0986	0.0047 ± 0.0029
Diabetes	StatCore base	4028.9521 ± 366.8942	50.4348 ± 1.0521	0.3095 ± 0.0986	0.0032 ± 0.0006
Diabetes	prototype compressed	3766.8309 ± 482.5068	50.6118 ± 1.6040	0.3540 ± 0.1103	0.0406 ± 0.0154
Diabetes	learnable metric	3583.6096 ± 538.1745	48.5502 ± 1.3647	0.3859 ± 0.1099	0.1990 ± 0.0146
Diabetes	statistical k selection	3665.1121 ± 534.7023	49.3741 ± 4.2763	0.3764 ± 0.0692	0.1007 ± 0.0099

reliability intervals are not provided by KNN. A stronger study would pre-register datasets and metrics, compare nested validation to a locked external test, include high-dimensional and imbalanced regimes, report peak memory, and correct for multiple benchmark comparisons.

IX. CONCLUSION

StatCore implements standard scaled Euclidean KNN and three concrete modifications: task-specific cluster prototypes, a bounded diagonal soft-neighbour metric, and a finite-grid exact LOO choice of k. This specification distinguishes those algorithms from textbook NCA and foldwise cross-validation where the implementation differs. Benchmark tables must be interpreted as seed-averaged measurements under the stated protocol, not evidence that a named variant is universally more accurate, calibrated, or reliable.

REFERENCES

[1] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.

[2] E. Fix and J. L. Hodges, "Discriminatory analysis: Nonparametric discrimination, consistency properties," USAF School of Aviation Medicine, Randolph Field, TX, Tech. Rep. 4, 1951.

[3] J. Goldberger, S. Roweis, G. Hinton, and R. Salakhutdinov, "Neighbourhood components analysis," in *Advances in Neural Information Processing Systems 17*, 2005, pp. 513–520.

[4] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, Mar. 1982.

[5] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: with Applications in R*, 2nd ed. Springer, 2021.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.